

C++ Programming Textbook Notes

Module: Pure Virtual Functions, Abstract Classes, and Interface Architecture

1. CONCEPTUAL FOUNDATIONS

Core OOP Architecture

What is a Pure Virtual Function?

A **Pure Virtual Function** in C++ is a virtual function declared in a base class that does not provide an implementation (definition) within that base class. It acts as an absolute contract that enforces all derived classes to override and implement the function.

Key Definition: Pure Virtual Function & Abstract Class

- **Pure Virtual Function:** A member function in a base class assigned `= 0` during declaration. It serves as a placeholder requiring mandatory definition in derived classes.
- **Abstract Class:** Any class containing at least one pure virtual function automatically becomes an **Abstract Class**. Instantiation (creating direct objects) of an abstract class is strictly forbidden by the compiler.

Syntax & Pure Specifier (`= 0`)

To declare a pure virtual function, append the pure specifier syntax `= 0` to the function prototype in the base class:

```
virtual return_type function_name(parameters) = 0;
```

Why Do We Need Abstract Classes?

In software engineering, base classes often represent generalized concepts rather than concrete entities (e.g., a generic Shape or GraphicObject). Creating an object of type Shape directly makes no logical sense because a "Shape" lacks a concrete form without being a Circle, Rectangle, or Triangle. Abstract classes enforce proper object-oriented hierarchy by serving strictly as base blueprints.

✓ Permitted Operations

- Declaring pointers to abstract class: `A* ptr;`
- Declaring references to abstract class: `A& ref = objB;`
- Inheriting from abstract base class

✗ Strictly Forbidden

- Direct object creation: `A obj;` (Compile Error)
- Allocation via new: `new A();` (Compile Error)
- Passing abstract class by value in parameters

Foundational Example (From Notes)

```
#include <iostream> using namespace std; class A { // Abstract Base Class (contains pure
```

Abstract Classes vs. Pure Interfaces

In C++, there is no separate interface keyword (unlike Java or C#). Instead, interfaces are constructed using abstract classes:

- **Abstract Class:** May contain member variables, concrete function definitions, AND at least one pure virtual function.
- **Pure Interface Class:** A class containing *ONLY* pure virtual functions, no member data, and a virtual destructor.

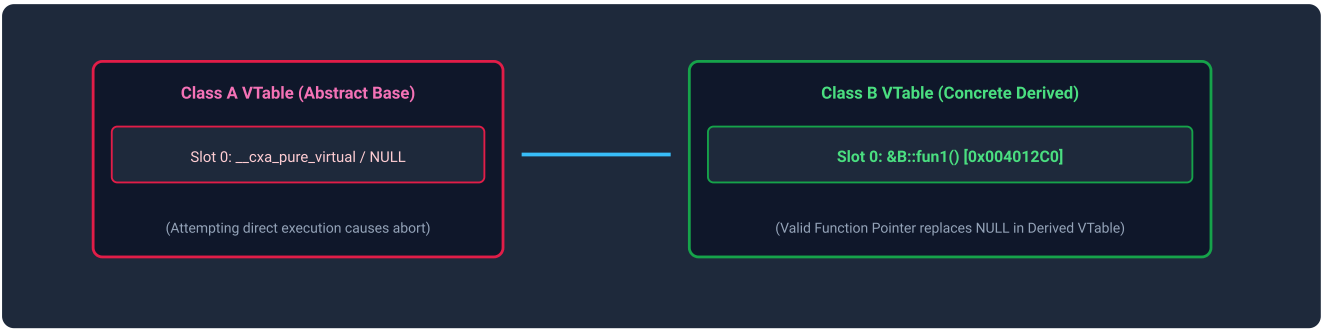
Comparison Matrix

Feature	Regular Concrete Class	Abstract Class	Pure Interface
Direct Instantiation	Allowed (A obj;)	Forbidden	Forbidden
Pure Virtual Methods	None	At least 1 method	All methods are pure virtual
Member Variables	Allowed	Allowed	Disallowed / No state
Primary Purpose	Concrete object implementation	Partial code reusability & blueprint	Strict API Contract definition

VTable Resolution for Pure Virtual Functions

How does the compiler handle a pure virtual function inside the class Virtual Table (vtable)?

When a class contains a pure virtual function, the compiler reserves a slot in the base class vtable. Since no body exists for the pure virtual function in the base class, the compiler puts a null pointer or points to a special runtime handler function like `__cxa_pure_virtual`.



Runtime Error - Pure Virtual Function Called:

If a pure virtual function is invoked indirectly during construction/destruction of an abstract base class, the program crashes with `pure virtual function call` (or `__cxa_pure_virtual`) because the derived slot has not been populated yet!

3. ADVANCED CODE WALKTHROUGH & MODERN C++ BEST PRACTICES

Professional
Implementation

Modern C++ Principles for Pure Virtual Functions

1. **Explicit override Keyword (C++11):** Always tag overridden implementations in derived classes with `override`. It catches typo errors in function signatures at compile-time.
2. **Virtual Destructor in Abstract Base Classes:** Always provide a `virtual ~Base() = default;` destructor to prevent dynamic deletion resource leaks.
3. **Pure Virtual Functions with Bodies:** Surprisingly, C++ allows pure virtual functions to have a definition outside the class body! This is commonly used for pure virtual destructors.

Comprehensive Production Code Example

```
#include <iostream> #include <memory> using namespace std; // Abstract Base Class acting
```

Key Summary Takeaways

Summary Checklist

1. **Abstract Class Creation:** Adding `= 0` to any virtual method turns the class abstract.
2. **Mandatory Overriding:** Derived classes MUST override all pure virtual functions to become concrete (instantiable) classes.
3. **Polymorphic Binding:** Base pointers (e.g., `Shape*`) can point to any derived object, enabling flexible polymorphic architecture.